23.1 Bash 执行引擎

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~120k tokens，输出 ~6k tokens（本节）

上一章我们全面了解了 OpenClaw 的工具系统——工具如何注册、如何执行、如何控制权限。从本章开始，我们深入到具体工具实现层面。Bash 执行引擎是 OpenClaw 中**最复杂、安全要求最高**的工具：它让 AI Agent 具备了在操作系统上执行任意命令的能力，但同时也带来了巨大的安全风险。本节从命令执行核心出发，逐步解析 PTY 终端模拟、PATH 管理以及安全审批等关键机制。

23.1.1 exec 工具的总体架构

源文件分布

Bash 工具的实现分布在多个文件中：

文件	职责
`bash-tools.ts`	桶文件（barrel），re-export exec 和 process 工具
`bash-tools.exec.ts`	**exec 工具核心**——命令执行引擎（~1630 行）
`bash-tools.process.ts`	process 工具——后台进程管理
`bash-tools.shared.ts`	共享工具函数（环境变量、工作目录、日志截取）
`bash-process-registry.ts`	进程注册表——内存中管理所有 exec 会话
`shell-utils.ts`	Shell 配置检测、二进制输出清理、进程树杀死
`pty-dsr.ts`	PTY DSR（Device Status Report）请求处理
`pty-keys.ts`	PTY 按键编码（send-keys、paste 操作）
`exec-approvals.ts` （infra/）	命令审批系统——白名单、安全策略评估

三种执行宿主模式

exec 工具支持三种执行宿主（Host），对应不同的安全级别：

```
┌──────────────┐  ┌──────────────┐  ┌──────────────┐
│   sandbox    │  │   gateway    │  │     node     │
│  (Docker)    │  │  (宿主机)    │  │  (远程节点)  │
├──────────────┤  ├──────────────┤  ├──────────────┤
│ 隔离容器执行 │  │ 直接在服务器 │  │ 通过 Gateway │
│ 安全性最高   │  │ 上执行       │  │ RPC 转发到   │
│ 默认模式     │  │ 需审批或白名单│  │ 伴侣应用执行 │
└──────────────┘  └──────────────┘  └──────────────┘
```

> **衍生解释**：在容器化（Containerization）技术中，Docker 是最流行的容器引擎。"sandbox" 模式下，OpenClaw 在一个 Docker 容器内执行命令，容器与宿主机文件系统和网络隔离，即使 AI 执行了恶意命令，也不会影响宿主系统。这是经典的**沙箱（Sandbox）**安全模型。

工具 Schema 定义

exec 工具通过 TypeBox 定义参数 Schema，LLM 可以理解并填入：

```
// src/agents/bash-tools.exec.ts — execSchema
const execSchema = Type.Object({
  command:   Type.String({ description: "Shell command to execute" }),
  workdir:   Type.Optional(Type.String()),    // 工作目录
  env:       Type.Optional(Type.Record(...)),  // 环境变量
  yieldMs:   Type.Optional(Type.Number()),     // 多久后转后台
  background:Type.Optional(Type.Boolean()),    // 立即后台化
  timeout:   Type.Optional(Type.Number()),     // 超时秒数
  pty:       Type.Optional(Type.Boolean()),    // PTY 模式
  elevated:  Type.Optional(Type.Boolean()),    // 提权执行
  host:      Type.Optional(Type.String()),     // sandbox|gateway|node
  security:  Type.Optional(Type.String()),     // deny|allowlist|full
  ask:       Type.Optional(Type.String()),     // off|on-miss|always
  node:      Type.Optional(Type.String()),     // 节点 ID
});
```

`yieldMs` 和 `background` 参数是 OpenClaw 的特色设计。命令执行可以先同步等待一段时间（默认 10 秒），如果命令尚未完成，就自动转入后台——这比简单的"同步阻塞到完成"或"立即后台化"更灵活。

23.1.2 命令执行核心流程

`createExecTool()` 工厂函数

exec 工具由工厂函数 `createExecTool(defaults?)` 创建。工厂接收默认配置参数（`ExecToolDefaults`），返回一个 `AgentTool` 实例。核心配置项包括：

```
// src/agents/bash-tools.exec.ts — ExecToolDefaults（简化）
type ExecToolDefaults = {
  host?: ExecHost;              // 默认执行宿主
  security?: ExecSecurity;      // 安全策略
  ask?: ExecAsk;                // 审批模式
  sandbox?: BashSandboxConfig;  // Docker 沙箱配置
  pathPrepend?: string[];       // PATH 前缀
  safeBins?: string[];          // 安全二进制列表
  backgroundMs?: number;        // 后台化等待时间
  timeoutSec?: number;          // 默认超时（秒）
  elevated?: ExecElevatedDefaults; // 提权配置
  sessionKey?: string;          // 会话标识
  // ...
};
```

execute 处理链

当 LLM 调用 exec 工具时，execute 函数依次执行以下步骤：

```
┌─ 1. 参数验证与规范化 ─┐
│  解析 command、workdir、env 等参数        │
│  处理 background/yieldMs 选项            │
└──────────────────────┘
           │
           ▼
┌─ 2. 宿主模式选择 ────┐
│  sandbox → Docker 容器执行               │
│  gateway → 宿主机直接执行（需审批）        │
│  node    → RPC 远程节点执行               │
└──────────────────────┘
           │
           ▼
┌─ 3. 安全检查 ────────┐
│  elevated（提权）验证                     │
│  环境变量安全校验                         │
│  allowlist + safeBins 白名单评估          │
│  是否需要用户审批?                        │
└──────────────────────┘
           │
           ▼
┌─ 4. 环境构建 ────────┐
│  sandbox: buildSandboxEnv()               │
│  gateway: 合并宿主 env + shell PATH       │
│  PATH 前缀注入                            │
└──────────────────────┘
           │
           ▼
┌─ 5. 进程创建 ────────┐
│  runExecProcess() 选择执行方式:           │
│  ├─ sandbox → docker exec                 │
│  ├─ PTY    → node-pty spawn               │
│  └─ 普通   → child_process spawn          │
└──────────────────────┘
           │
           ▼
┌─ 6. 输出收集与超时 ──┐
│  stdout/stderr 实时收集                   │
│  超时自动 SIGKILL                         │
│  yieldMs 到期后自动转后台                 │
└──────────────────────┘
```

23.1.3 三种进程创建方式

`runExecProcess()` 根据配置选择不同的进程创建策略：

策略一：Docker 沙箱执行

```
// src/agents/bash-tools.exec.ts — runExecProcess sandbox 分支（简化）
if (opts.sandbox) {
  const { child } = await spawnWithFallback({
    argv: [
      "docker",
      ...buildDockerExecArgs({
        containerName: opts.sandbox.containerName,
        command: opts.command,
        workdir: opts.containerWorkdir ?? opts.sandbox.containerWorkdir,
        env: opts.env,
        tty: opts.usePty,
      }),
    ],
    options: {
      cwd: opts.workdir,
      detached: process.platform !== "win32",
      stdio: ["pipe", "pipe", "pipe"],
    },
    fallbacks: [{ label: "no-detach", options: { detached: false } }],
  });
  child = spawned;
}
```

关键点：

- `buildDockerExecArgs()` 构建 `docker exec -i [-t] -w <dir> -e KEY=VAL ... <container> sh -lc '<command>'` 命令
- 使用 `-l`（login shell）确保 `.profile` 等初始化脚本被加载，从而获得完整的 PATH
- `spawnWithFallback()` 提供降级机制——如果 `detached` 模式 spawn 失败（某些 Node.js 版本/平台限制），自动回退到非 detached 模式

> **衍生解释**：`detached: true` 是 Node.js `child_process.spawn()` 的选项，表示子进程与父进程分离运行。在 Unix 系统上，这会创建一个新的进程组（Process Group），使得父进程退出时子进程不会被杀死。OpenClaw 需要这个特性来支持后台长时间运行的命令。

策略二：PTY 模拟终端执行

```
// src/agents/bash-tools.exec.ts — runExecProcess PTY 分支（简化）
} else if (opts.usePty) {
  const { shell, args: shellArgs } = getShellConfig();
  try {
    const ptyModule = await import("@lydell/node-pty");
    const spawnPty = ptyModule.spawn ?? ptyModule.default?.spawn;

    pty = spawnPty(shell, [...shellArgs, opts.command], {
      cwd: opts.workdir,
      env: opts.env,
      name: process.env.TERM ?? "xterm-256color",
      cols: 120,
      rows: 30,
    });

    // 构造 stdin 适配器——PTY 用 pty.write() 而非 child.stdin
    stdin = {
      write: (data, cb) => { pty?.write(data); cb?.(null); },
      end: () => {
        const eof = process.platform === "win32" ? "\x1a" : "\x04";
        pty?.write(eof);
      },
    };
  } catch (err) {
    // PTY 创建失败 → 降级为普通 child_process spawn
    warnings.push(`PTY spawn failed; retrying without PTY.`);
    // 回退到策略三...
  }
}
```

> **衍生解释**：PTY（Pseudo Terminal，伪终端）是操作系统内核提供的一种机制，它模拟了一个终端设备。许多命令行程序（如 `vim` 、 `htop` 、 `less` ）会检测自己是否运行在终端中——如果不是（比如 stdout 是管道），它们会改变行为（禁用颜色、不显示 TUI 界面、甚至拒绝运行）。通过 PTY，OpenClaw 可以让这些程序认为自己运行在一个真正的终端里。

PTY 模式有几个特殊之处：

1. **终端尺寸**：设置为 120 列 × 30 行，这是典型的终端尺寸
2. **TERM 环境变量**：默认为 `xterm-256color` ，支持 256 色输出
3. **stdin 适配**：PTY 不使用 Node.js 的 `child.stdin` ，而是通过 `pty.write()` 写入

4. **EOF 信号**：Unix 用 `\x04` （Ctrl-D），Windows 用 `\x1a` （Ctrl-Z）
5. **自动降级**：如果 `node-pty` 模块不可用（未安装或平台不支持），自动回退到普通 spawn

策略三：普通子进程执行

```
// src/agents/bash-tools.exec.ts — 普通 spawn 分支
} else {
  const { shell, args: shellArgs } = getShellConfig();
  const { child } = await spawnWithFallback({
    argv: [shell, ...shellArgs, opts.command],
    options: {
      cwd: opts.workdir,
      env: opts.env,
      detached: process.platform !== "win32",
      stdio: ["pipe", "pipe", "pipe"],
    },
    fallbacks: [{ label: "no-detach", options: { detached: false } }],
  });
}
```

最简单的模式：获取系统 Shell（通过 `getShellConfig()` ），然后用 `-c` 参数执行命令。

23.1.4 PTY 数据流处理

PTY 模式下，数据流的处理与普通 child_process 有显著差异。

DSR 请求过滤

```
// src/agents/bash-tools.exec.ts — PTY 数据处理（简化）
if (pty) {
  const cursorResponse = buildCursorPositionResponse();
  pty.onData((data) => {
    const raw = data.toString();
    const { cleaned, requests } = stripDsrRequests(raw);
    if (requests > 0) {
      for (let i = 0; i < requests; i++) {
        pty.write(cursorResponse);  // 回复每个 DSR 查询
      }
    }
    handleStdout(cleaned);  // 清理后的数据才进入输出缓冲
  });
}
```

> **衍生解释**：DSR（Device Status Report）是终端控制序列标准（ANSI/VT100）的一部分。当程序向终端发送 `ESC[6n` 时，它期望终端回复当前光标位置（格式为 `ESC[行;列R`）。像 `vim`、`zsh` 等程序在启动时会发送 DSR 来获取光标位置。OpenClaw 的 PTY 层需要"假装"自己是终端来回复这些查询，否则程序会一直等待回复而卡住。

DSR 处理的实现非常精练，只有 16 行：

```
// src/agents/pty-dsr.ts — 完整实现
const ESC = String.fromCharCode(0x1b);
const DSR_PATTERN = new RegExp(`${ESC}\\[\\??6n`, "g");

export function stripDsrRequests(input: string) {
  let requests = 0;
  const cleaned = input.replace(DSR_PATTERN, () => {
    requests += 1;
    return "";
  });
  return { cleaned, requests };
}

export function buildCursorPositionResponse(row = 1, col = 1): string {
  return `\x1b[${row};${col}R`;
}
```

输出格式对比

特性	普通模式	PTY 模式
stdout/stderr 分离	✓ 独立流	✗ 混合在一起
ANSI 颜色码	通常无	包含
DSR 查询	无	需要过滤和回复
TUI 程序支持	不支持	支持
数据事件	`child.stdout.on('data')`	`pty.onData()`

23.1.5 环境变量安全机制

宿主模式环境变量黑名单

当执行宿主为 `gateway` （宿主机）或 `node` （远程节点）时，OpenClaw 会严格检查 Agent 请求的环境变量：

```ts
// src/agents/bash-tools.exec.ts — 危险环境变量黑名单
const DANGEROUS_HOST_ENV_VARS = new Set([
  "LD_PRELOAD",              // 共享库注入
  "LD_LIBRARY_PATH",         // 自定义库搜索路径
  "LD_AUDIT",                // 审计共享库
  "DYLD_INSERT_LIBRARIES",   // macOS 库注入
  "DYLD_LIBRARY_PATH",       // macOS 库搜索路径
  "NODE_OPTIONS",            // Node.js 启动选项注入
  "NODE_PATH",               // Node.js 模块搜索路径
  "PYTHONPATH",              // Python 模块搜索路径
  "PYTHONHOME",              // Python 安装路径
  "RUBYLIB",                 // Ruby 库路径
  "PERL5LIB",                // Perl 库路径
  "BASH_ENV",                // Bash 启动时自动执行的脚本
  "ENV",                     // sh 启动时执行的脚本
  "GCONV_PATH",              // glibc 字符转换路径
  "IFS",                     // Shell 内部字段分隔符
  "SSLKEYLOGFILE",           // TLS 密钥泄露
]);

const DANGEROUS_HOST_ENV_PREFIXES = ["DYLD_", "LD_"];
```

> **衍生解释**： `LD_PRELOAD` 攻击是 Linux 系统上一种经典的代码注入技术。 `LD_PRELOAD` 环境变量指定一个共享库（.so 文件），这个库会在所有其他库之前被加载。攻击者可以用自定义库覆盖标准库函数（如 `read()` 、 `write()` ），从而拦截或篡改程序行为。类似地，macOS 上的 `DYLD_INSERT_LIBRARIES` 具有相同的危害。

验证函数会拒绝任何尝试设置这些变量的请求：

```typescript
function validateHostEnv(env: Record<string, string>): void {
  for (const key of Object.keys(env)) {
    const upperKey = key.toUpperCase();
    // 前缀匹配（如 LD_xxx、DYLD_xxx）
    if (DANGEROUS_HOST_ENV_PREFIXES.some(p => upperKey.startsWith(p))) {
      throw new Error(`Security Violation: '${key}' is forbidden.`);
    }
    // 精确匹配
    if (DANGEROUS_HOST_ENV_VARS.has(upperKey)) {
      throw new Error(`Security Violation: '${key}' is forbidden.`);
    }
    // PATH 也不允许自定义
    if (upperKey === "PATH") {
      throw new Error(`Security Violation: Custom 'PATH' is forbidden.`);
    }
  }
}
```

sandbox（Docker）模式跳过此检查——因为容器内的环境变量不会影响宿主机。

PATH 管理策略

不同模式下的 PATH 处理各不相同：

```
sandbox 模式：
  buildSandboxEnv() → 使用沙箱配置的 DEFAULT_PATH

gateway 模式：
  1. 从宿主进程 process.env 获取基础 PATH
  2. 如果用户未指定 PATH → 调用 getShellPathFromLoginShell()
     获取 login shell 中的完整 PATH（包含 nvm、rbenv 等工具链）
  3. 使用 applyPathPrepend() 注入配置的前缀路径

node 模式：
  由远程节点自行管理 PATH
```

Safe Bins（安全二进制白名单）

```
// src/infra/exec-approvals.ts — 默认安全二进制列表
export const DEFAULT_SAFE_BINS = [
  "jq", "grep", "cut", "sort", "uniq", "head", "tail", "tr", "wc"
];
```

在 `allowlist` 安全模式下，如果命令是这些"安全"工具之一（且不带文件路径参数），可以直接执行而不需要审批。 `isSafeBinUsage()` 函数做了精细的检查：

- 命令的可执行文件名必须在 `safeBins` 集合中
- 可执行文件必须解析到具体路径（防止 PATH 注入）
- 参数中不能包含文件路径（防止读写敏感文件）

23.1.6 命令审批系统

审批文件结构

命令审批的配置存储在 `~/.openclaw/exec-approvals.json` ：

```json
{
  "version": 1,
  "socket": {
    "path": "~/.openclaw/exec-approvals.sock",
    "token": "base64url-random-token"
  },
  "defaults": {
    "security": "allowlist",
    "ask": "on-miss"
  },
  "agents": {
    "pi": {
      "security": "allowlist",
      "ask": "on-miss",
      "allowlist": [
        { "id": "uuid", "pattern": "/usr/bin/git", "lastUsedAt": 1700000000 },
        { "id": "uuid", "pattern": "/usr/local/bin/npm" }
      ]
    }
  }
}
```

三层安全策略

策略	security	ask	行为
最严格	`deny`	—	拒绝一切执行
白名单	`allowlist`	`on-miss`	匹配白名单则执行，否则询问用户
全开放	`full`	`off`	直接执行（仅适用于沙箱）

`minSecurity()` 函数确保安全等级只能**降低**（取最小值）：

```typescript
// src/infra/exec-approvals.ts
export function minSecurity(a: ExecSecurity, b: ExecSecurity): ExecSecurity {
  const order = { deny: 0, allowlist: 1, full: 2 };
  return order[a] <= order[b] ? a : b;
}
```

Shell 命令分析引擎

在评估白名单时，OpenClaw 需要**解析 Shell 命令**来提取可执行文件路径。这是一个迷你的 Shell 解析器：

```
// src/infra/exec-approvals.ts — 命令分析流程（简化）
export function evaluateShellAllowlist(params) {
  // 1. 拆分链式命令（&&, ||, ;）
  const chainParts = splitCommandChain(params.command);

  // 2. 对每个链段，拆分管道（|）
  for (const part of chainParts) {
    const pipelineSplit = splitShellPipeline(part);

    // 3. 对每个管道段，tokenize 并解析可执行文件
    for (const segment of pipelineSplit.segments) {
      const argv = tokenizeShellSegment(segment);
      const resolution = resolveCommandResolution(argv);
      // resolution.resolvedPath → 如 "/usr/bin/git"
    }
  }

  // 4. 每个段都必须匹配白名单或 safeBins
  return { allowlistSatisfied, allowlistMatches };
}
```

命令解析器支持：

- 链式操作符：`&&` 、 `||` 、 `;`
- 管道：`|`
- 单引号和双引号
- 转义字符 `\`
- 拒绝的危险 token：`>` 、 `<` 、 `` ` `` 、 `$()` 、 `\n`

审批流程

当命令不在白名单中且 `ask` 不为 `off` 时，触发用户审批：

```
Agent 发起 exec → 命令不在白名单
    │
    ▼
创建 approval 请求 → 通过 Gateway 发送给用户
    │
    ▼
等待用户决策（最长 120 秒）
    │
    ├─ "allow-once"  → 执行此次
    ├─ "allow-always" → 执行此次 + 加入白名单
    └─ "deny"        → 拒绝执行

如果超时：
    └─ 根据 askFallback 决定
       ├─ "full"      → 允许执行
       ├─ "allowlist"  → 仅白名单匹配则允许
       └─ "deny"      → 拒绝
```

审批请求通过 Unix Domain Socket 传递，确保只有本机进程能参与审批。

23.1.7 后台化与产出收集

yieldMs 机制

exec 工具最巧妙的设计之一是 `yieldMs` ——在命令执行期间，先同步等待一段时间，如果命令没完成就自动转入后台：

```
// src/agents/bash-tools.exec.ts — yield 机制（简化）
if (allowBackground && yieldWindow !== null) {
  if (yieldWindow === 0) {
    // background: true → 立即后台化
    markBackgrounded(run.session);
    return { status: "running", sessionId: run.session.id };
  } else {
    // yieldMs 到期后转后台
    yieldTimer = setTimeout(() => {
      markBackgrounded(run.session);
      resolve({ status: "running", sessionId: ... });
    }, yieldWindow);
  }
}

// 如果命令在 yieldMs 内完成
run.promise.then((outcome) => {
  if (yielded) return;  // 已经后台化了，不再处理
  clearTimeout(yieldTimer);
  resolve({ status: outcome.status, aggregated: outcome.aggregated });
});
```

这个设计的好处：

- 快速命令（如 `ls` 、 `cat` ）直接返回结果，无需额外的 poll 调用
- 长时间命令（如 `npm install` 、 `make` ）不会阻塞 Agent 的对话循环
- Agent 可以在等待时继续处理其他任务，然后用 `process` 工具检查结果

超时处理

```
if (opts.timeoutSec > 0) {
  timeoutTimer = setTimeout(() => {
    timedOut = true;
    killSession(session);        // 先 SIGKILL 进程
    timeoutFinalizeTimer = setTimeout(() => {
      finalizeTimeout();         // 1 秒后强制结算
    }, 1000);
  }, opts.timeoutSec * 1000);
}
```

超时后的处理分两步：

1. 立即杀死进程（通过 `killProcessTree()` 杀掉整个进程树）
2. 等待 1 秒让内核清理，然后标记会话为 `failed` 并收集已有输出

本节小结

1. **exec 工具是 OpenClaw 最复杂的工具**，约 1630 行代码，涉及进程管理、安全审批、PTY 终端等多个子系统
2. **三种执行宿主**：sandbox（Docker 隔离）、gateway（宿主机直接执行）、node（远程节点 RPC）
3. **三种进程创建方式**：Docker exec、PTY spawn（node-pty）、普通 child_process spawn，每种都有降级策略
4. **PTY 模式需要处理 DSR 请求**——伪装为终端回复光标位置查询，否则 TUI 程序会卡住
5. **多层安全防护**：环境变量黑名单、PATH 修改禁止、白名单 + safe bins、用户审批流程
6. **Shell 命令解析器**支持链式操作、管道、引号，可以精确提取每个管道段的可执行文件路径
7. **yieldMs 设计**让快速命令直接返回、长时间命令自动后台化，兼顾响应速度和非阻塞性